"One fact base, many audiences -- clarity under pressure is a deliverable, too."

MODULE 47

Professional Communication and Collaboration

Communicate like a professional engineer: write blameless incident updates, reviewable PR descriptions, and audience-aware stakeholder emails, give concrete peer-review feedback, and ship a consistent, secret-free communication packet.


TEKsystems
Global Services
WORKFORCE
DEVELOPMENT


TEKsystems
Global Services
WORKFORCE
DEVELOPMENT

MODULE 47 OVERVIEW

Write blameless incident updates, reviewable PR descriptions, and audience-aware stakeholder emails from one shared fact base -- then prove it with a peer review.

The Northstar CRM SEV-2 incident from Module 46 needs to be communicated to four different audiences -- responders, engineers, reviewers, and stakeholders -- without ever contradicting itself.

DURATION & LAB

3 Hours Theory

Audience-aware communication, Agile roles, documentation, code and design reviews, escalation, and presenting solutions.

90 Min Lab

lab47-crm: incident update, technical follow-up, PR description, stakeholder email, and peer review for the Northstar CRM release.

Scenario

The same SEV-2 crm-api 1.4.0 incident from Module 46 -- now told to four different audiences.

MODULE ARC

One Fact Base

Confirmed facts, assumptions, and unknowns -- written once, reused everywhere.

Audience-Aware Writing

Incident updates, PR descriptions, and stakeholder emails, each tailored without changing facts.

Peer Review & Packet

Concrete rewrites, blameless tone, and a secret-free communication packet.

KEY TAKEAWAY Total time: 3 hours theory + 90 minutes hands-on lab = 4.5 hours. Clarity under time pressure is a deliverable equal to code -- blame, invented causes, and leaked secrets are unacceptable.

WHY PROFESSIONAL SKILLS MATTER

Engineers spend as much time communicating as coding -- and miscommunication during an incident causes real, measurable damage.

Time Well Spent

Standups, chat, PR comments, incident channels, and design docs consume real engineering hours.

Zero Tolerance for Blame

Blame, invented root cause, and leaked credentials are unacceptable, full stop.

One Fact Base

Different audiences need different depth -- but never contradictory facts.

Graded Like Code

This module's lab is scored on a 100-point rubric as rigorous as any coding lab.

KEY TAKEAWAY Clarity under time pressure is a deliverable equal to code -- a peer must be able to brief from your packet without ever asking 'which version is true?'

Why Professional Skills Matter

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to communicate technical work clearly, honestly, and appropriately for any audience.

1. Adapt tone and detail to audience -- without ever changing the underlying facts.
2. Write factual, blameless incident updates with a mandatory next-update time.
3. Create reviewable PR descriptions with verification evidence and a rollback plan.
4. Translate technical risk into plain-language business impact.
5. Separate confirmed facts, assumptions, and unknowns in every document.
6. Give specific, respectful peer-review feedback with concrete rewrite suggestions.
7. Scrub secrets and personal data from every communication artifact.
8. Align CI/CD and Kafka evidence from Labs 43-46 into one coherent narrative.

KEY TAKEAWAY These eight objectives are drawn directly from the graded lab -- you will practice every one of them in lab47-crm's five communication artifacts.

COMMUNICATION IN SOFTWARE TEAMS

Software teams communicate through many channels, each with its own audience contract -- and the same facts must read differently, but never contradict, across all of them.

COMMUNICATION CHANNELS

Standups and chat -- synchronous, brief, status-focused.

PR comments and code review -- asynchronous, precise, line-cited.

Incident channels -- urgent, factual, blameless.

Design docs and ADRs -- durable, reasoned, reviewable.

Retrospectives -- team-only, improvement-focused.

AUDIENCE NEEDS

- Responders need symptoms, impact, and the next-update time.
- Engineers need the change, evidence, and the rollback plan.
- Reviewers need PR verification steps and named risk.
- Stakeholders need business impact and an ETA, without jargon.

Technical vs. business language, same fact

```
Technical: roll back the crm-api digest, watch consumer lag.
Business:  we are restoring the previous version and monitoring closely.
```

KEY TAKEAWAY One shared fact base, many audience-tailored documents -- never re-derive or restate facts differently per channel.

Communication in Software Teams

GOAL

Clear, timely, and effective communication that enables collaboration, alignment, and delivering high-quality software.

1. Clear & Open Communication

- Be clear, concise, and specific
- Ask questions and seek clarity
- Encourage open feedback
- Share information transparently

2. Right Channel, Right Time

- Use the right channel for the right message
- Urgent vs. non-urgent communication
- Avoid overloading channels
- Respect focus time

3. Active Listening

- Listen to understand, not to reply
- Acknowledge and confirm
- Value different perspectives
- Summarize key points

4. Collaboration & Alignment

- Align on goals, priorities, and expectations
- Keep everyone in the loop
- Document decisions and action items
- Use shared tools and docs

5. Constructive Feedback

- Be specific and actionable
- Focus on behavior and impact
- Balance positives and improvements
- Give and welcome feedback

6. Context & Documentation

- Provide context, not just tasks
- Document important discussions
- Keep knowledge accessible
- Reduce repetition and confusion

7. Communication Best Practices

Regular Check-ins
Keep the team synchronized.

Stand-ups That Work
Share updates, not status reports.

Follow-ups & Ownership
Confirm, track, and close the loop.

Assume Positive Intent
Be respectful and solution-focused.

Psychological Safety
Encourage everyone to speak up.

RESULT

Stronger Teamwork | Fewer Misunderstandings | Faster Delivery | Better Quality | Happier Team

STAKEHOLDER COMMUNICATION

Stakeholder emails must lead with outcome and user impact, translate jargon into plain language, and stay consistent with the shared fact base.

TEMPLATE TO FILL

Subject: _____

What customers/agents see: _____

What we are doing: _____

When next update: _____

What we need from you: _____

RULES

- Lead with outcome and impact, not implementation detail.
- Translate jargon (DLT, digest) into plain language or a short gloss.
- Check every claim against the shared fact base before sending.
- Scrub token-looking strings and real names or emails.

Worked example (CRM 1.4 release email)

```
Subject: CRM 1.4 release planned for Tuesday, 18:00 UTC
Engineering completed tests and staging verification and will monitor
error rate and response time for 60 minutes after deployment.
```

KEY TAKEAWAY Audience-inappropriate dumps of Kafka internals to executives lose core marks -- lead with outcome, not implementation.

Technical vs Business Communication

Aspect	Technical Communication	Business Communication
Purpose	To explain, instruct, or inform about technical topics or processes	To persuade, inform, collaborate, or build professional relationships
Audience	Specialized audience with technical knowledge or specific interest	Internal or external stakeholders, clients, colleagues, managers, general audience
Content	Technical data, instructions, manuals, reports, specifications, procedures	Business ideas, strategies, policies, reports, proposals, presentations, emails
Language	Formal, precise, technical, and industry-specific	Clear, professional, and audience-appropriate
Tone	Objective, factual, and impersonal	Polite, persuasive, and relationship-oriented
Goal	To convey complex information accurately and effectively	To achieve business objectives and foster understanding or action
Examples	User manuals, technical reports, process documents, FAQs	Emails, business letters, proposals, meetings, presentations, memos

Stakeholder Communication

WHY IT MATTERS

- Builds trust and credibility
- Aligns expectations
- Reduces risks and conflicts
- Supports better decisions
- Drives project and organizational success

KEY PRINCIPLES

Be Clear
Use simple, jargon-free language.

Be Timely
Share information when it matters.

Be Transparent
Share both successes and challenges.

Be Respectful
Value different perspectives and feedback.

COMMUNICATION CHANNELS

- Emails
- Meetings & Workshops
- Presentations
- Instant Messaging
- Reports & Dashboards
- Portals & Intranets
- Newsletters & Updates

STAKEHOLDER TYPES

Sponsors
Provide resources and overall direction.

Team Members
Execute work and provide status and feedback.

Customers/Users
Receive value and provide requirements and feedback.

Partners/Vendors
Support delivery and bring specialized expertise.

Regulators/Community
Ensure compliance and represent broader interests.

COMMUNICATION PROCESS

1. Identify
Identify stakeholders and their needs.

→

2. Plan
Plan messages, channels, and timing.

→

3. Communicate
Deliver messages clearly and consistently.

→

4. Engage
Listen, respond, and encourage feedback.

→

5. Evaluate
Monitor results and improve continuously.

TRY IT YOURSELF -- EXERCISE 4: STAKEHOLDER TODOS

Reinforces: translating a technical incident into a plain-language stakeholder email before the graded lab requires the full packet.

STEPS (notes/lab47-stakeholder-todos.md)

Step	What To Do
1 -- Fill Template	Fill subject, what customers/agents see, actions, next update, and any ask.
2 -- Plain Language	Translate jargon such as DLT or digest into plain language.
3 -- Consistency Check	Confirm the email matches the shared fact base -- no contradictions.
4 -- Scrub	Remove any token-looking strings or real emails before saving.

Template to fill

```
Subject: _____   What customers/agents see: _____
What we are doing: _____   When next update: _____   What we need from you: _____
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 4 before continuing: exercises/exercise-04-stakeholder-todos.md (workspace: examples/module-47-exercises).

AGILE TEAM STRUCTURES & SCRUM ROLES

Agile teams organize around short feedback loops -- Scrum and Kanban are the two most common structural frameworks for that rhythm.

Role	Responsibility	Communicates With
Product Owner	Owns priorities and the backlog.	Stakeholders and the team.
Scrum Master	Facilitates process and removes blockers.	Team and leadership.
Development Team	Delivers the working increment.	Product Owner and QA.
Release Commander	Owns incident and release decisions.	Ops, engineers, and stakeholders.

SCRUM CEREMONY CYCLE

1

1. Sprint Planning

Commit to a scoped, achievable set of work.

2

2. Daily Standup

Status, next action, and blockers -- briefly.

3

3. Sprint Review

Demo completed work to stakeholders for feedback.

4

4. Retrospective

Review what went well and what to improve.

KEY TAKEAWAY Kanban visualizes flow with a board that limits work in progress; Scrum timeboxes flow into sprints -- both exist to keep communication regular and predictable.

Agile Team Structures

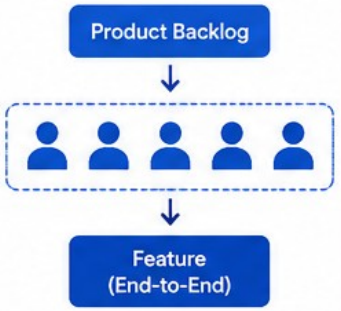
Cross-Functional Team

Description
A self-contained team with all skills needed to deliver end-to-end functionality.

Best For
Small to medium products with well-defined goals.

Key Characteristics
- End-to-end ownership
- All necessary skills
- High collaboration
- Fast decision making
- Strong accountability

Feature Team

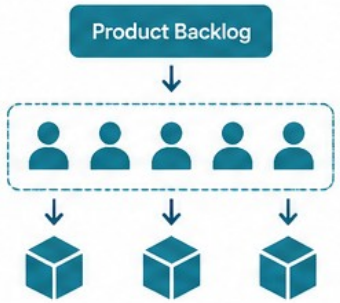


Description
A dedicated team focused on delivering a specific feature from start to finish.

Best For
When features are large and require a dedicated team.

Key Characteristics
- One team per feature
- Clear scope and goal
- End-to-end delivery
- Focused and independent
- High ownership

Component Team

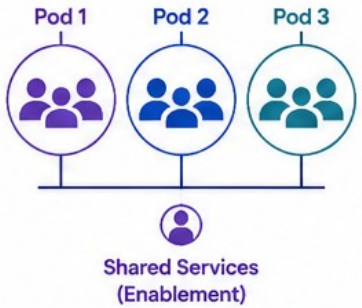


Description
Teams responsible for specific components or layers of the system.

Best For
Large systems with complex architecture.

Key Characteristics
- Specialized by component
- Deep technical expertise
- Reusable components
- Strong standards focus
- May create dependencies

Pod Structure

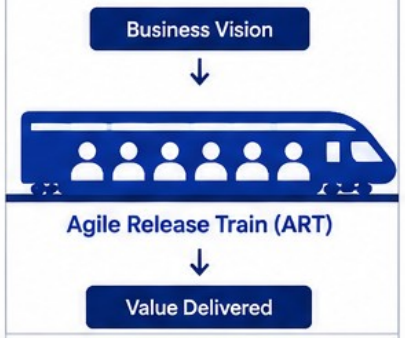


Description
Small, cross-functional teams (pods) with alignment through shared services.

Best For
Scaling agile across multiple related teams.

Key Characteristics
- Small, independent pods
- Shared services model
- Autonomous with alignment
- Scalable and flexible
- Clear interfaces

Agile Release Train (ART)



Description
A large group of agile teams aligned to deliver value incrementally.

Best For
Large enterprises and complex solutions.

Key Characteristics
- Multiple teams (50–125 people)
- Common goals and roadmap
- PI Planning and cadence
- Shared backlog and DevOps
- End-to-end value delivery

Benefits of Agile Team Structures

- Better collaboration
- Faster delivery
- Higher quality
- Greater adaptability
- Stronger ownership

Scrum Roles and Responsibilities

	Product Owner	Scrum Master	Developers (Scrum Team)
ROLE			
FOCUS	Maximize value of the product and work of the Scrum Team	Ensure the Scrum Team understands and follows Scrum	Deliver a potentially releasable product increment
KEY RESPONSIBILITIES	✓ Define and communicate the product vision and goals ✓ Manage the Product Backlog ✓ Prioritize backlog items based on value and customer needs ✓ Ensure backlog items are clear, transparent, and understood ✓ Collaborate with stakeholders and the Scrum Team ✓ Accept or reject work based on acceptance criteria ✓ Ensure the product delivers maximum value	✓ Facilitate Scrum events ✓ Remove impediments and unblock the team ✓ Coach the team on Scrum principles, practices, and values ✓ Promote a collaborative and self-organizing environment ✓ Help the team improve continuously ✓ Protect the team from external distractions ✓ Ensure transparency and effective communication	✓ Commit to and achieve Sprint Goals ✓ Plan and estimate work ✓ Build high-quality product increments ✓ Collaborate and support each other ✓ Adapt and re-plan as needed ✓ Ensure continuous improvement in process and product ✓ Take ownership of the work and meet commitments

Shared Responsibility — All roles work together to deliver value, inspect progress, adapt, and improve continuously while upholding the Scrum values: Commitment, Courage, Focus, Openness, and Respect.

DAILY STANDUPS & SPRINT REVIEWS

Standups and sprint reviews are both short, structured communication rituals -- one is internal and daily, the other is external and periodic.

DAILY STANDUP FORMAT

What I did yesterday -- one or two sentences.

What I'll do today -- specific, not vague.

Blockers -- named specifically, not 'stuck on stuff'.

SPRINT REVIEW VS. RETROSPECTIVE

- Sprint Review -- demo completed work to stakeholders for feedback.
- Retrospective -- team-only review of what went well and what to improve.
- Both exist to catch drift early, before it becomes a bigger problem.

The standup-to-incident-update pattern

```
Status  ->  Next action  ->  Next check time
(the exact same three-part discipline an incident update requires)
```

KEY TAKEAWAY A good standup and a good incident update share the same skeleton: status, next action, and a concrete next-check or next-update time.

Daily Standups

A short, time-boxed daily meeting for the Scrum Team to synchronize work, create a plan for the day, and remove impediments.

PURPOSE

- Inspect progress toward the Sprint Goal
- Synchronize work and identify dependencies
- Surface and plan to remove impediments
- Adapt the plan for the next 24 hours

THE 3 QUESTIONS

1. **What did I do yesterday?**
   Share your progress since the last standup.
2. **What will I do today?**
   Share your plan for the next 24 hours.
3. **What impediments are blocking me?**
   Share any blockers or challenges you need help with.

GUIDELINES

- Keep it short: 15 minutes or less
- Stand up (keeps it focused and fast)
- Same time and place every day
- Be concise and stick to the 3 questions
- Focus on the team, not on individuals
- Update details outside the standup
- Action items are taken outside the meeting

BEST PRACTICES

- Encourage everyone to participate
- Stay focused on the 3 questions
- Time-box the meeting
- Take conversations offline
- Track impediments to closure

HOW IT WORKS

Team gathers (standing up) → Everyone answers the 3 questions → Impediments identified → Plan for the day is aligned

WHO ATTENDS

Scrum Team Only

Developers, Scrum Master, and Product Owner

KEY TAKEAWAY

Daily standups build transparency, alignment, and commitment—helping the team deliver value every day.

Sprint Reviews

A collaborative meeting at the end of the Sprint to inspect the Increment, gather feedback, and adapt the product.

PURPOSE

- Inspect the product Increment
- Gather feedback from stakeholders
- Adapt the Product Backlog based on feedback
- Demonstrate value and transparency

DETAILS

When
At the end of the Sprint

Timebox
Max 4 hours for a 1-month Sprint (usually 1–2 hours)

Where
Same place as Sprint Planning (Virtual or In-person)

Who Facilitates
Scrum Master

Who Attends
Scrum Team and Stakeholders

WHAT HAPPENS

1. **Set the Stage**
   Scrum Master explains the purpose and agenda.
2. **Demo the Increment**
   Developers demonstrate what was built and how it works.
3. **Gather Feedback**
   Stakeholders ask questions and share their feedback and suggestions.
4. **Adapt the Backlog**
   Product Owner updates and prioritizes the Product Backlog based on feedback.
5. **Close the Review**
   Summarize key takeaways and next steps.

STAKEHOLDERS

Customers / Users
Provide feedback on value and usability

Product Owner
Shares vision and gathers input for backlog

Team
Demonstrates work and answers questions

Other Stakeholders
Business, Marketing, Sales, Ops, etc.

GUIDELINES

- Focus on the Increment, not the process
- Encourage open and honest feedback
- Keep it collaborative and constructive
- Avoid long status updates or deep technical dives
- Timebox the meeting and stay on track

OUTPUT

- Stakeholder Feedback and Suggestions
- Updated and Prioritized Product Backlog
- Shared Understanding of Product Value and Progress

BENEFITS

- Ensures the right product is being built
- Increases stakeholder engagement
- Improves product quality and value
- Enables frequent feedback and adaptation
- Builds trust and transparency

KEY TAKEAWAY | Sprint Reviews ensure we build the right product by delivering value, learning continuously, and adapting to change.

TECHNICAL DOCUMENTATION

Documentation is the persistent, audience-independent record that prevents the 'which version is true?' problem this whole module is built to avoid.

WHY DOCUMENTATION MATTERS

It is the durable record everyone can reference later.

A peer can brief from it without asking what's true.

ADR -- a short note explaining an architecture choice and why.

RFC -- a proposal document for feedback before deciding.

THE THREE-WAY SPLIT

- Confirmed -- what you know for certain, with evidence.
- Assumptions -- what you believe but haven't verified.
- Unknowns -- what genuinely isn't known yet (e.g., root cause).

shared-facts.md skeleton

```
## Confirmed
- Symptom:   - Start (UTC):   - Mitigation in progress:
## Assumptions
## Unknowns
- Root cause:
## Owners  ## Next update
```

KEY TAKEAWAY A peer can brief from the packet without asking 'which is true?' -- that is the standard every piece of documentation in this module must meet.

Importance of Technical Documentation

Technical documentation is the foundation of successful software development, operations, and maintenance. It ensures knowledge is captured, shared, and preserved for current and future users.

1. Improves Communication
Creates a common understanding among developers, stakeholders, and users.

2. Saves Time and Effort
Helps teams find information quickly, reducing repeated questions and rework.

3. Ensures Consistency
Provides standards, guidelines, and references to maintain consistency across projects.

4. Reduces Risks
Well-documented systems are easier to support, secure, and audit, reducing operational risks.

5. Onboards Faster
New team members and users ramp up quickly with clear, well-structured documentation.

6. Simplifies Maintenance
Easier to understand, troubleshoot, and enhance systems over time.

7. Supports Scalability
Good documentation supports growth, new features, and changing technologies.

8. Preserves Knowledge
Captures decisions, designs, and context to prevent knowledge loss when people move on.

GOOD TECHNICAL DOCUMENTATION IS:

Accurate
Correct and up-to-date information.

Clear
Easy to understand with simple, concise language.

Complete
Covers all necessary details and use cases.

Well-Structured
Organized logically with headings, sections, and links.

Accessible
Easy to find and available to the right audience.

Maintainable
Reviewed and updated regularly to stay relevant.

Writing Technical Documentation

Clear. Accurate. Complete. Consistent.

1 Plan & Understand Your Audience

- Identify the purpose and scope.
- Know your audience and their needs.
- Gather requirements and relevant information.

2 Organize Your Content

- Create an outline or structure.
- Use logical flow and grouping.
- Follow a standard template if available.

3 Write Clearly and Concisely

- Use simple, precise language.
- Write in short sentences.
- Prefer active voice and present tense.

4 Use Visuals Effectively

- Add diagrams, charts, and screenshots.
- Ensure visuals are labeled and easy to understand.

5 Provide Examples & Code

- Include examples for clarity.
- Provide code snippets with explanations.
- Show input and expected output.

6 Review & Validate

- Review for accuracy, completeness, and consistency.
- Validate steps and examples.
- Get peer or SME review if possible.

7 Publish & Maintain

- Publish in the right format and location.
- Keep documentation updated with changes.
- Collect feedback and improve continuously.

Best Practices

Be User-Focused
Write for your audience, not for yourself.

Be Accurate
Ensure all information is correct and verifiable.

Be Consistent
Use consistent terms, style, and formatting throughout.

Be Complete
Cover all necessary details without unnecessary clutter.

Be Easy to Navigate
Use headings, lists, tables, and TOC for easy navigation.

Be Maintainable
Keep content modular and easy to update and reuse.

Good Documentation

- Reduces support and training effort
- Improves user experience
- Ensures consistency and reliability
- Speeds up onboarding
- Supports maintenance and scalability

> Good documentation is not just a deliverable—it's a product that helps users succeed.

TRY IT YOURSELF -- EXERCISE 1: BUILD SHARED FACT BASE

Reinforces: separating confirmed facts, assumptions, and unknowns before writing any audience-specific document.

STEPS (notes/lab47-fact-base.md)

Step	What To Do
1 -- Audience Table	Note what each audience needs: Responders, Engineers, Reviewers, Stakeholders.
2 -- Scenario Values	Record SEV-2, HTTP 503 symptom, crm-api 1.4.0, and the CRM fixtures.
3 -- Three Lists	Split content into Confirmed / Assumed / Unknown -- explicitly.
4 -- Mitigation Stub	Add a mitigation note: roll back toward 1.3.2; watch readiness and lag.

Audience needs -- quick reference

```
Responders: symptoms, impact, next-update time.
Engineers: change, evidence, rollback plan.
Reviewers: PR verification steps, named risk.
Stakeholders: business impact and ETA, no jargon.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 1 before continuing: exercises/exercise-01-fact-base.md (workspace: examples/module-47-exercises).

ARCHITECTURE & API DOCUMENTATION

Two specialized documentation types -- one explains a design decision, the other defines a contract between systems.

Term	Meaning
ADR	Architecture Decision Record -- a short note explaining one choice and why.
RFC	Request for Comments -- a proposal document for feedback before deciding.
API Documentation	Defines how one program calls another -- paired with tests and a rollback plan.
Architecture Documentation	Explains the system's structure and links dashboards/runbooks by path.

GOOD DOCUMENTATION HABITS

One Decision, One ADR

Keep each architecture decision record scoped to a single choice.

RFC Before Building

Circulate a proposal for feedback before implementation starts.

API Docs Ship With Tests

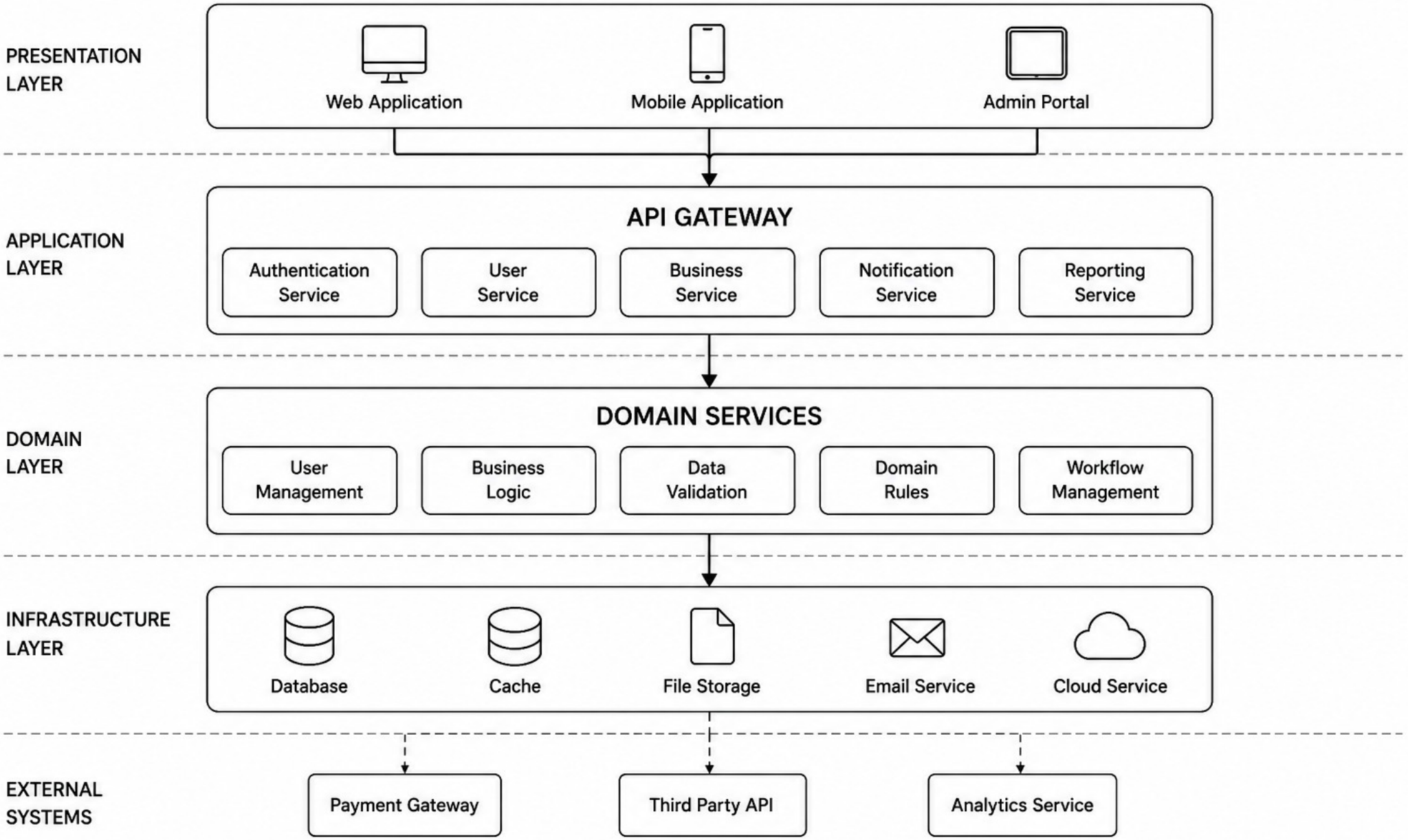
Pair the contract with verification evidence, not just a description.

Link, Don't Paste

Reference dashboards and runbooks by path -- never paste credentials.

KEY TAKEAWAY Architecture documentation should link dashboards and runbooks without ever exposing secrets -- reference the path, not the credential.

ARCHITECTURE DOCUMENTATION



API DOCUMENTATION

DOCUMENTATION SECTIONS

- Overview
- Authentication
- Endpoints
- Request & Response
- Errors
- Examples
- Rate Limits
- Webhooks
- FAQ
- Changelog

1. OVERVIEW

Welcome to the API.

This API provides programmatic access to our resources and services.

Base URL: `https://api.example.com/v1`

2. AUTHENTICATION

All requests require authentication using API Key in the header.

```
Header:
x-api-key: your_api_key
```

3. ENDPOINTS

METHOD	ENDPOINT	DESCRIPTION
GET	/users	Retrieve a list of users
POST	/users	Create a new user
GET	/users/{id}	Retrieve a user by ID
PUT	/users/{id}	Update a user by ID

4. REQUEST & RESPONSE

Request Example

```
GET /users/123 HTTP/1.1
Host: api.example.com
x-api-key: your_api_key
Accept: application/json
```

Response Example (200 OK)

```
{
   "id": 123,
   "name": "John  Doe",
   "email": "john.doe@example.com"
}
```

5. ERRORS

Errors follow standard HTTP status codes.

Example (404 Not Found)

```
{
   "error": "Not Found",
   "message": "User not found",
   "code": 404
}
```

6. EXAMPLES

Code examples in multiple languages.

cURL | JavaScript | Python | Java

7. RATE LIMITS

API rate limits:

- 1000 requests per hour
- 100 requests per minute

Headers:

```
X-RateLimit-Limit
X-RateLimit-Remaining
X-RateLimit-Reset
```

8. WEBHOOKS

Real-time notifications sent to your endpoint for specific events.

Example Events:

- user.created
- user.updated
- user.deleted

9. FAQ

- How do I get an API key?
- What are the rate limits?
- How do I handle errors?
- Where can I get support?

10. CHANGELOG

Track API updates and revisions.

v1.2.0 – 2024-05-01
- Added /users/{id} endpoint

v1.1.0 – 2024-03-15
- Rate limit increased

CODE REVIEWS & CONSTRUCTIVE FEEDBACK

Peer review is the QA gate for words, the same way code review is the QA gate for code -- feedback must be specific and paired with a concrete rewrite.

THE REWRITE RULE

Cite the exact line or section under review.

Propose a concrete rewrite, not just a criticism.

'Looks good' and 'unclear' are not acceptable feedback.

Design reviews follow the same rule for proposed approaches.

AUTHOR OWNS THE CALL

- The author records a disposition: accept or decline, with rationale.
- Feedback is not automatically applied -- the author decides.
- Reviewer focus: two or three precise questions, not a general pass.

Before -> After (the exercise 5 pattern)

```
Before: "Someone broke Kafka again."
After:  factual, blameless restatement of what changed and its effect.
Before: "Fixed stuff."
After:  specific verify and rollback language.
```

KEY TAKEAWAY 'Looks good' and 'unclear' are not feedback -- every review comment needs a cited line and a concrete rewrite.

CODE REVIEWS

1 CODE CHANGE

Developer writes code and commits changes to the repository.

2 PULL REQUEST

Developer creates a pull request (PR) with description of changes and context.

3 AUTOMATED CHECKS

CI/CD pipeline runs automated checks, builds, tests, and linting.

4 CODE REVIEW

Reviewers examine code for correctness, quality, security, performance, and best practices.

5 FEEDBACK & DISCUSSION

Reviewers provide feedback. Developer addresses comments and updates code.

6 APPROVAL & MERGE

Once approved, the PR is merged into the main branch.

CODE REVIEW BEST PRACTICES

FOCUS ON CODE, NOT THE AUTHOR

Provide constructive feedback and be respectful.

BE CONSISTENT

Follow team standards and coding guidelines consistently.

FOCUS ON WHAT MATTERS

Prioritize issues that impact correctness, security, or maintainability.

KEEP IT SMALL

Small PRs are easier to review and reduce cognitive load.

BE TIMELY

Review and respond promptly to keep momentum.

LEARN & IMPROVE

Use reviews as a learning opportunity for the whole team.

BENEFITS

- Improves code quality and consistency
- Reduces bugs and security risks
- Shares knowledge across the team
- Maintains a healthy and scalable codebase

DESIGN REVIEWS

A collaborative evaluation of a design to ensure quality, feasibility, and alignment with requirements.

1 PREPARE

- Define objectives
- Identify stakeholders
- Distribute designs
- Prepare review checklist

2 CONDUCT REVIEW

Review the design, ask questions, discuss alternatives, and evaluate against criteria.

3 EVALUATE

Assess the design for correctness, completeness, consistency, risks, and compliance with standards and requirements.

4 DOCUMENT FINDINGS

Record issues, risks, decisions, and action items with severity and owner.

5 RESOLVE ISSUES

Design team addresses issues and updates the design as needed.

6 FOLLOW UP

Verify changes, close action items, and obtain review closure.

KEY REVIEW CRITERIA

FUNCTIONAL

Meets requirements and use cases.

TECHNICAL

Technically sound, feasible, and maintainable.

ARCHITECTURAL

Aligns with architecture, patterns, and standards.

SECURITY

Addresses security requirements and risks.

PERFORMANCE

Meets performance, scalability, and capacity needs.

COST

Cost-effective and resource efficient.

COMPLIANCE

Complies with regulations, policies, and guidelines.

PARTICIPANTS

- Design Team
- Developers
- Architects
- Product Owner / Business Analyst
- QA / Test Team
- Other Stakeholders

BENEFITS

- Improves design quality and reliability.
- Identifies risks and issues early.
- Aligns stakeholders and expectations.
- Reduces rework and project costs.
- Ensures compliance with standards and best practices.

TRY IT YOURSELF -- EXERCISE 5: PEER REVIEW REWRITE PRACTICE

Reinforces: turning vague or blaming language into specific, respectful, actionable feedback.

STEPS (notes/lab47-peer-review-practice.md)

Step	What To Do
1 -- Blame Rewrite	Rewrite "Someone broke Kafka again" as a blameless, factual statement.
2 -- PR Rewrite	Rewrite "Fixed stuff" into specific verify and rollback language.
3 -- Cite the Line	Make sure each rewrite cites the line or section being reviewed.
4 -- Save Notes	Write both before/after pairs to notes/lab47-peer-review-practice.md.

The two rewrites to produce

```
Before: "Someone broke Kafka again."   ->   After: _____
Before: "Fixed stuff."   ->   After: _____
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 5 before continuing: exercises/exercise-05-peer-review-practice.md (workspace: examples/module-47-exercises).

OWNERSHIP & TECHNICAL DEBT

Taking ownership means standing behind what you write -- and naming technical debt honestly is the same habit applied to code, not documentation, deferred forever.

TAKING OWNERSHIP

The author's disposition (accept/decline) is the final word on feedback.

Own the accuracy of every artifact you write, not just the code.

Failure handling -- correcting and owning mistakes -- is a graded rubric category.

Hiding a problem is worse than reporting it honestly.

MANAGING TECHNICAL DEBT

- Debt is a shortcut or deferred decision that makes future work riskier.
- Naming it explicitly is itself a professional communication act.
- The 'Unknowns' bucket in shared-facts.md is the same honesty habit.
- Escalate debt before it becomes an incident, not after.

Honest labeling, not confident guessing

```
"Root cause remains under investigation."
(Not: "It was a network issue." -- unverified and stated as fact is a failure.)
```

KEY TAKEAWAY Ownership and honest debt-labeling are the same skill: say what you actually know, say what you don't, and never fill the gap with a confident guess.

MANAGING TECHNICAL DEBT

A continuous process to identify, prioritize, and reduce technical debt while delivering value.

1 IDENTIFY
Identify technical debt through code reviews, tools, monitoring, and team feedback.

2 CATEGORIZE
Categorize debt by type, component, and impacted area.

3 PRIORITIZE
Assess impact vs. effort and prioritize based on business value, risk, and urgency.

4 PLAN
Plan debt reduction alongside feature work. Allocate time and resources.

5 ADDRESS
Refactor, simplify, or replace to reduce debt incrementally and safely.

6 MONITOR
Continuously monitor debt levels and prevent new debt from accumulating.

TECHNICAL DEBT CATEGORIES

- **Code Debt** — Complex, duplicated, or poorly structured code
- **Design Debt** — Flawed architecture, poor abstractions, tight coupling
- **Infrastructure Debt** — Outdated tools, environments, or configurations
- **Test Debt** — Missing tests, flaky tests, low coverage
- **Security Debt** — Known vulnerabilities, outdated dependencies, weak practices
- **Process Debt** — Inefficient workflows, poor documentation, lack of standards

STRATEGIES TO MANAGE TECHNICAL DEBT

- Pay Down Continuously – Allocate regular capacity to reduce debt.
- Boy Scout Rule – Leave the codebase cleaner than you found it.
- Refactor Opportunistically – Improve code when making changes.
- Timebox Debt Work – Fix within a defined time and scope.
- Automate – Use tools, linters, and tests to enforce quality.

BEST PRACTICES

- Make debt visible and transparent.
- Track debt with metrics and dashboards.
- Align debt work with business goals.
- Communicate and involve the whole team.
- Prevent new debt through standards and reviews.

THE CONTINUOUS CYCLE

BENEFITS

- Improved Code Quality & Reliability
- Faster Delivery & Agility
- Lower Maintenance Costs
- Reduced Risk & Incidents
- Higher Team Productivity & Satisfaction
- Better Scalability & Sustainability

ESCALATION AND RISK COMMUNICATION

An incident update is this module's model escalation artifact -- factual, blameless, and anchored by a mandatory next-update time.

Field	Example Value
Severity	SEV-2 (example severity level).
Impact	Some agents see HTTP 503 opening customer profiles.
Current Action	Rolling back crm-api 1.4.0 toward 1.3.2; watching readiness and lag.
Known	Error rate increased after rollout; database health remains green.
Unknown	Root cause remains under investigation.
Owner	Release commander -- Team A (a role title, never a person's name).
Next Update	A concrete time, e.g. 15:00 UTC, or sooner if impact changes.

ESCALATION RULES

No Blame

Never name a person or claim an unsupported cause.

UTC Always

Every timestamp uses UTC -- no local-time ambiguity.

Next Update Mandatory

Missing it is a named failure experiment: ops ambiguity.

Role Titles Only

Write 'Release commander,' never a person's real name.

KEY TAKEAWAY Quantify impact only where you have evidence; use role titles, not names; and never publish an incident update without a next-update time.

ESCALATION AND RISK COMMUNICATION

Timely escalation and clear communication to manage risks and enable informed decisions.

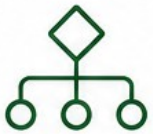
1 IDENTIFY RISK

Detect potential risk early through monitoring, reviews, or feedback.

2 ASSESS IMPACT

Evaluate likelihood, impact, urgency, and affected areas.

3 DETERMINE ESCALATION



Determine if escalation is required based on thresholds and impact.

4 ESCALATE

Escalate to the right person or team with complete context.

5 COMMUNICATE

Communicate clearly, transparently, and regularly.

6 TRACK & RESOLVE

Track actions, resolve the risk, and close the communication loop.

EFFECTIVE RISK COMMUNICATION PRINCIPLES

BE CLEAR

Use clear, simple language and avoid technical jargon.

BE CONCISE

Share only relevant information that supports decisions.

BE TRANSPARENT

Share facts, impact, and uncertainties openly.

BE TIMELY

Communicate early and update as the situation evolves.

BE ACTIONABLE

Provide clear options, recommendations, and next steps.

BE ACCOUNTABLE

Own the message and follow through on commitments.

KEY STAKEHOLDERS & COMMUNICATION CHANNELS

STAKEHOLDER	PURPOSE	CHANNELS
Team Members	Aware of risks, impacts, and actions	Team Meetings, Chat, Email, Dashboards
Team Leads / Managers	Make decisions and provide support	Meetings, Email, Reports
Project Sponsors / Leadership	High-level visibility and decisions	Status Reports, Escalation Reports
Customers / Clients	Understand impact and expectations	Client Updates, Calls, Emails

ESCALATION MATRIX (EXAMPLE)

RISK LEVEL	IMPACT	URGENCY	ESCALATE TO	RESPONSE TIME
CRITICAL	High (Severe)	Immediate	Executive / Sponsor	Within 1 Hour
HIGH	High (Major)	Urgent	Manager / Director	Within 4 Hours
MEDIUM	Medium (Moderate)	Soon	Team Lead	Within 1 Business Day
LOW	Low (Minor)	Normal	Team	As Planned

RISK COMMUNICATION FLOW

Risk Occurs
Identify and document the risk

Assess & Analyze
Assess impact, likelihood, and urgency

Escalate
Escalate to the appropriate stakeholder

Communicate
Share risk, impact, options, and next steps

Act & Resolve
Execute actions, mitigate risk, resolve issues

Close the Loop
Confirm resolution and communicate closure

TRY IT YOURSELF -- EXERCISE 2: INCIDENT UPDATE SKELETON

Reinforces: writing a factual, blameless incident update with every mandatory field present.

STEPS (notes/lab47-incident-update.md)

Step	What To Do
1 -- Six Sections	Draft summary, impact, current status, known facts, next update, contacts.
2 -- Rule Check	Confirm no invented root cause, no credentials, no PII anywhere.
3 -- Impact Sentence	Write one impact sentence naming agents and synthetic customer IDs only.
4 -- Next Update	Set a concrete next-update time -- never leave it blank.

Fields that must never be missing

```
Severity   Impact   Current Action   Known / Unknown   Owner (role title)   Next Update
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 2 before continuing: exercises/exercise-02-incident-update.md (workspace: examples/module-47-exercises).

PRESENTING TECHNICAL SOLUTIONS

A pull request description is this module's model for presenting a technical solution -- it must justify, describe, prove, and plan for rollback.

Section	What Goes There
Why	State the customer or operational problem being solved.
What Changed	Backend, messaging/data, and deployment/configuration, listed separately.
Verification	A table: tests, security checks, happy and failure paths -- pass or fail.
Risk and Rollback	Compatibility, observability, and the exact rollback action.
Reviewer Focus	Two or three precise questions for the reviewer to focus on.

COMMON PR ANTI-PATTERNS

Vague Description

"Fixed stuff" tells a reviewer nothing to verify.

Missing Verification

No evidence the change was actually tested.

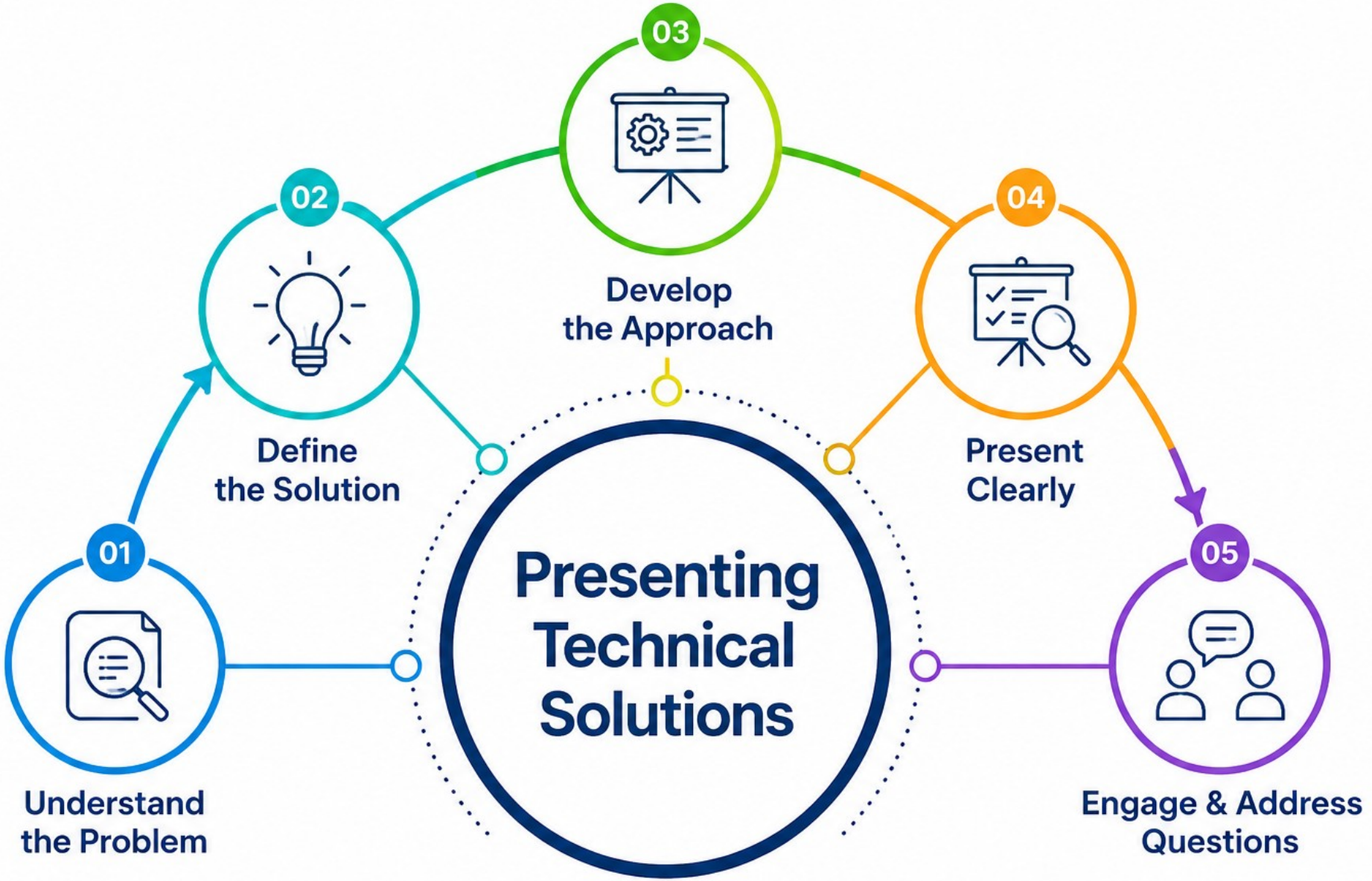
No Rollback Plan

Add a concrete digest or command reference.

Unscoped Changes

Mixing backend, config, and messaging changes with no separation.

KEY TAKEAWAY No rollback plan is a named failure mode -- add concrete digest or command references so a reviewer never has to ask 'how would we undo this?'



TRY IT YOURSELF -- EXERCISE 3: PR DESCRIPTION OUTLINE

Reinforces: outlining a reviewable pull request -- the full packet comes later in Lab 47.

STEPS (notes/lab47-pr-description.md)

Step	What To Do
1 -- Five Sections	Draft Why, What Changed, Verification, Rollback, and Reviewer Focus.
2 -- Verify Bullets	Write two verification bullets using the synthetic CRM fixtures.
3 -- Rollback Detail	Name the exact rollback action -- a digest or command, not a promise.
4 -- Reviewable Check	Confirm a reviewer would not need to ask questions in chat first.

Verification command referenced by the lab

```
./mvnw -B test   (or a link to the CI run)
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 3 before continuing: exercises/exercise-03-pr-description.md (workspace: examples/module-47-exercises).

ARCHITECTURE WALKTHROUGHS & DEMOS

The same underlying facts get presented completely differently to engineers than to stakeholders -- walkthroughs go deep, demos lead with outcome.

ARCHITECTURE WALKTHROUGH (ENGINEERS)

Walk the timeline of what happened, in order.

Present the signals -- CI digest, Kafka lag, error rate.

State hypotheses, actions taken, and results observed.

Distinguish correlation from causation explicitly.

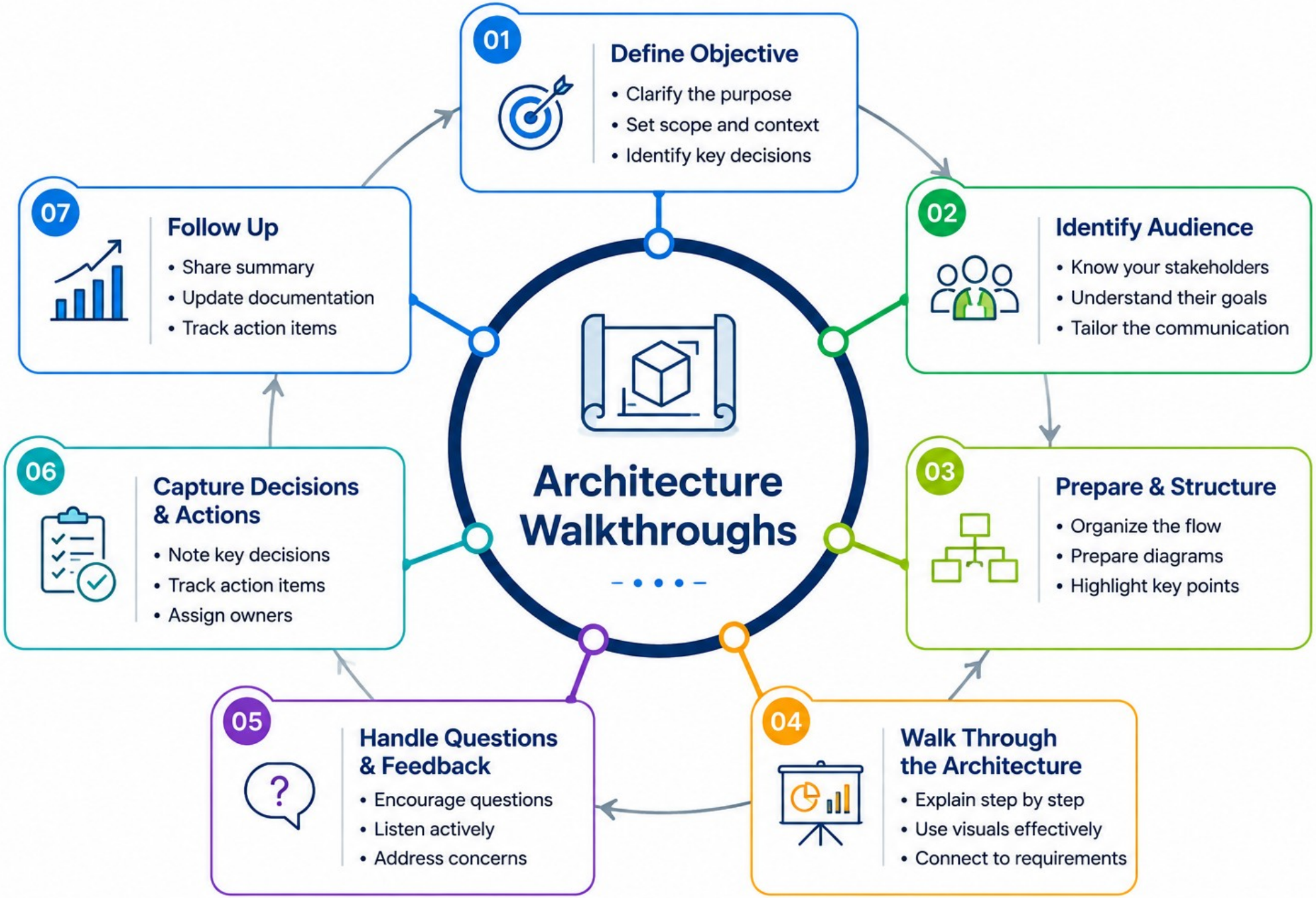
DEMO (STAKEHOLDERS)

- Lead with outcome and user impact, not implementation.
- Avoid detail that doesn't support a decision.
- Give a clear next step or ask, if any.
- Bonus skill: a 60-second executive briefing.

Same fact, two audiences

```
Engineer:    consumer lag on crm-customer-projection-v1 spiked after the 1.4.0 rollout.
Stakeholder: the release is being monitored closely and rolled back if needed.
```

KEY TAKEAWAY Link dashboards and runbooks without exposing secrets, and never state correlation as causation -- 'happened after' is not the same as 'was caused by'.



PROFESSIONAL CONDUCT & COLLABORATION

Most communication failures on a team are not writing failures -- they're process failures: no shared fact sheet, or editing from memory instead of from source.

Symptom	Fix
Docs disagree on severity	Rebuild the fact sheet; re-sync every document from it.
Stakeholder panic	Lead with impact and actions, not raw uncertainty.
Conflicting next-update times	Keep a single source of truth in shared-facts.md.
Statuspage vs. Slack mismatch	Always edit from shared facts first, never draft twice.
Oversharing fixture emails	Use synthetic customer IDs only in any external-facing text.

CONDUCT RULES

No Blame

Never name a person or claim unsupported cause.

No Secrets

No tokens, connection strings, or personal emails, anywhere.

Role Titles

Prefer 'Release commander' over a person's real name.

One Fact Base

Every document is derived from it -- never redrafted independently.

KEY TAKEAWAY The central collaboration practice: one shared fact base drives every audience-specific document, reconciled through peer review before anything ships.

TRY IT YOURSELF -- EXERCISE 6: COMMUNICATION PACKET INDEX

Capstone: index the four Lab 47 artifacts and check them for consistency -- the final gate before Lab 47 begins.

STEPS (notes/lab47-packet-index.md)

Step	What To Do
1 -- List Artifacts	List the four artifacts: incident update, follow-up, PR, stakeholder email.
2 -- Consistency Checklist	Confirm same severity, start time, suspected change, and fixtures.
3 -- Evidence Links	Add placeholder links to CI/CD and Kafka evidence from Labs 43-46.
4 -- Scope Check	Confirm this is an index only -- writing the full packet is Lab 47 itself.

The four artifacts to index

```
1. Incident update    2. Technical follow-up
3. PR description    4. Stakeholder email + peer review notes
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 6 before continuing: exercises/exercise-06-packet-index.md (workspace: examples/module-47-exercises). Final gate before Lab 47.

LAB OVERVIEW -- NORTHSTAR STAKEHOLDER PACK

Write four audience-specific communication artifacts about the Module 46 SEV-2 incident, reconciled through a real peer review.

BUSINESS SCENARIO

- The same SEV-2 incident from Module 46 -- crm-api 1.4.0 causing HTTP 503 errors for some agents, mitigated by rolling back toward 1.3.2 while watching readiness and Kafka lag -- now needs to be communicated to four different audiences.
- Different audiences need different depth, but never contradictory facts. Blame, invented root cause, and leaked credentials are unacceptable.
- Clarity under time pressure is a deliverable equal to code -- a peer must be able to brief from your packet without ever asking 'which version is true?'

LEARNING OBJECTIVES

- Adapt tone and detail to audience without changing the underlying facts.
- Write a factual, blameless incident update with a mandatory next-update time.
- Create a reviewable PR description with verification and a rollback plan.
- Translate technical risk into plain-language business impact.
- Give specific, respectful peer-review feedback with rewrite suggestions.
- Scrub secrets and personal data from every artifact in the packet.

WHAT YOU BUILD

- lab47-crm under ~/java-bootcamp/examples/: communications/shared-facts.md, incident-update.md, technical-follow-up.md, pull-request-description.md, stakeholder-release-email.md, and peer-review.md, plus docs/release-briefing-notes.md.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan) and CUS-1002 (Ravi Singh) are synthetic -- never real customer PII. Duration: about 45 minutes with starters, up to 3-4 hours full path. Intermediate difficulty.

LAB TASKS AND DELIVERABLES

lab47-crm -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Collect shared facts: confirmed, assumptions, unknowns, owners, next update.
2	Define audience and purpose for each of the four documents before drafting.
3	Write the incident update: blameless, factual, with a mandatory next-update time.
4	Write the technical follow-up: timeline, signals, hypotheses, actions, results.
5	Draft the PR description: why, what changed, verification, risk and rollback.
6	Draft the stakeholder email: outcome-first, plain language, consistent with facts.
7	Run a peer review: cite lines, propose rewrites, record accept/decline dispositions.
8	Finalize the packet and run at least three failure experiments as evidence.

EXPECTED DELIVERABLES

- communications/incident-update.md
- communications/pull-request-description.md
- communications/stakeholder-release-email.md
- communications/peer-review.md
- Release briefing notes and shared facts; a consistent, secret-free packet.

RUBRIC (100 MARKS)

- Environment/Structure 10 * Core Impl 30 * Integration/Config 15 * Failure Handling 15 * Verification 10 * Security/Production Awareness 10 * Docs 10

KEY TAKEAWAY Invented root cause stated as fact loses failure/security marks. Secrets in the packet must be remediated. Dumping Kafka internals on executives loses core marks.

MODULE 47 SUMMARY

In this module, we explored how one shared fact base becomes four audience-specific documents, reconciled through honest, specific peer review.

KEY CONCEPTS COVERED

Shared Fact Base

Confirmed, assumptions, and unknowns -- written once, reused everywhere.

Incident Updates

Factual, blameless, and always anchored by a next-update time.

PR Descriptions

Why, what changed, verification, and an exact rollback plan.

Stakeholder Communication

Outcome-first, plain language, never contradicting the facts.

Peer Review

Cited lines, concrete rewrites, and an author's final disposition.

Professional Conduct

No blame, no invented cause, no secrets -- ever.

MODULE FLOW RECAP

1. Gather Facts
2. Draft Per Audience
3. Verify & Rollback
4. Peer Review
5. Ship the Packet

KEY TAKEAWAY Communication is a graded engineering skill: one fact base, many honest audiences, and a packet a peer can brief from without ever asking what's true.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of shared fact bases, incident updates, PR descriptions, and peer review.

1. What are the three categories every shared fact base must separate?

A. Bugs, Features, and Tasks

B. Confirmed, Assumptions, and Unknowns

C. High, Medium, and Low priority

D. Frontend, Backend, and Database

2. Which field is mandatory in every incident update, and its absence is a named failure?

A. Root cause

B. Blame assignment

C. Next-update time

D. Server hostname

3. Which five sections make up this module's PR description template?

A. Why, Changed, Verify, Rollback, Reviewer Focus

B. Title, Body, Labels, Assignee, Milestone

C. Summary, Diff, Tests, Approvals, Merge

D. Intro, Design, Code, Tests, Deploy

4. What should replace vague feedback like 'Looks good' or 'unclear' in a peer review?

A. A thumbs-up reaction and nothing else

B. A cited line with a concrete before/after rewrite

C. A private message to the author instead

D. Silence -- approve it anyway

KEY TAKEAWAY A fact base splits confirmed/assumptions/unknowns; incident updates require a next-update time; PR descriptions need five specific sections; feedback needs a cited rewrite.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on fixtures, unverified claims, secrets, and cross-document consistency.

5. What should stakeholder-facing text use instead of real customer names and emails?

A. Nothing -- omit the customer entirely

B. Internal ticket numbers only

C. Synthetic customer IDs like CUS-1001

D. The engineer's own name

6. What is the correct response to an unsupported claim about an incident's root cause?

A. State it confidently since it seems likely

B. Move it to the Unknowns list until confirmed

C. Delete the incident update entirely

D. Ask a stakeholder to guess instead

7. What must be scrubbed from every communication artifact before it ships?

A. Code comments

B. Tokens, connection strings, and personal emails

C. Test coverage numbers

D. Sprint velocity figures

8. What is the relationship between a stakeholder email and an incident update mid-incident?

A. They may state different severities if written by different people

B. They must stay consistent with the same shared facts

C. Only the incident update needs to be accurate

D. The stakeholder email should be written first, facts fixed later

KEY TAKEAWAY Use synthetic IDs, never real PII; unverified causes go to Unknowns; scrub secrets always; every document must agree with the same shared facts.

"One fact base, many audiences -- clarity under pressure is a deliverable, too."

MODULE 47 COMPLETE

Professional Communication and Collaboration

You can now write factual, blameless incident updates, reviewable PR descriptions, and audience-aware stakeholder emails, give specific peer-review feedback, and assemble a consistent, secret-free communication packet -- all from one shared fact base.